MySQL Introduction & Basics

1. What is Data?

Data is a collection of raw facts and figures.
Example:

- Name: Neeraj
- Age: 23
- City: Lucknow

By itself, data has no meaning until it is processed.

2. What is a Database?

A **database** is an organized collection of data stored electronically so that it can be easily accessed, managed, and updated.

Example:

A **student database** may contain:

- Student ID
- Name
- Class
- Marks

3. What is DBMS (Database Management System)?

A **DBMS** is software that helps users:

- Store data
- Retrieve data
- Update data
- Delete data

Examples of DBMS:

- MySQL
- Oracle
- SQL Server
- PostgreSQL

4. What is SQL?

SQL (Structured Query Language) is a standard language used to:

- Communicate with databases

- Store data
- Retrieve data
- Update data
- Delete data

Why SQL is used?

- It is easy to learn
- It works with large amounts of data
- It is supported by almost all databases

What is MySQL?

MySQL is an **open-source Relational Database Management System (RDBMS)** that uses **SQL** to manage data.

In simple words:

- **MySQL = Database software**
- **SQL = Language used to work with MySQL**

Why MySQL is called RDBMS?

Because MySQL stores data in the form of **tables** and maintains **relationships** between tables.

Example:

- Student table
- Course table

Both tables can be connected using a common key.

History of MySQL

- **Invented by:** Michael Widenius (Monty Widenius)
- **Developed by:** MySQL AB (Swedish company)
- **First released:** 1995
- **Currently owned by:** Oracle Corporation

Why MySQL was created?

- To provide a **fast**, **reliable**, and **free** database system
- To support **web applications**

Features of MySQL

- Open source (free)

- Fast performance
- Secure
- Easy to use
- Supports large databases
- Used in web development

Where is MySQL used?

- Websites (Facebook, YouTube – earlier versions)
- Web applications
- Banking systems
- College & school management systems
- E-commerce websites

Installing MySQL on Computer (Windows)

Step-1:

Go to official website:
https://dev.mysql.com/downloads/

Step-2:

Download **MySQL Installer for Windows**

Step-3:

Choose **Developer Default** or **Server Only**

Step-4:

Set:

- Root password
- Port number (default: 3306)

Step-5:

Install **MySQL Workbench**

What is MySQL Workbench?

MySQL Workbench is a **GUI tool** used to:

- Write SQL queries
- Create databases
- Create tables
- Manage MySQL server

How to Run First SQL Query

Open MySQL Workbench

Login with **root password**

Create Database:

```
CREATE DATABASE mydb;
```

Use Database:

```
USE mydb;
```

Show Databases:

```
SHOW DATABASES;
```

What is a Data Type?

A **data type** defines:

- What kind of data can be stored in a column
- How much space it will take

Example:

- Name → text
- Age → number
- Date → date

Data Types in MySQL

MySQL data types are mainly divided into **3 categories**:

1. Numeric Data Types

Used to store numbers.

Data Type	Description
INT	Stores whole numbers
FLOAT	Stores decimal numbers
DOUBLE	Stores large decimal numbers
DECIMAL	Stores exact decimal values
TINYINT	Stores very small integers

2. String (Character) Data Types

Used to store text.

Data Type	Description
CHAR	Fixed-length string
VARCHAR	Variable-length string
TEXT	Long text data
ENUM	List of predefined values

3. Date & Time Data Types

Used to store date and time.

Data Type	Description
DATE	Stores date (YYYY-MM-DD)
TIME	Stores time
DATETIME	Stores date & time
TIMESTAMP	Stores current date & time

Example Table Using Data Types

```
CREATE TABLE student (
    id INT,
    name VARCHAR(50),
    age INT,
    dob DATE
);
```

CRUD Operations in MySQL

What is CRUD?

CRUD stands for the four basic operations performed on data in a database:

Letter	Operation	Meaning
C	Create	Insert new data
R	Read	Retrieve data
U	Update	Modify existing data
D	Delete	Remove data

CRUD operations are the foundation of database management.

Example Table Used for CRUD

```
CREATE TABLE students (
```

```
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    course VARCHAR(50)
);
```

1. CREATE Operation (INSERT)

The **CREATE** operation is used to **insert new records** into a table.

Insert single record

```
INSERT INTO students (id, name, age, course)
VALUES (1, 'Neeraj', 23, 'B.Tech');
```

Insert multiple records

```
INSERT INTO students (id, name, age, course)
VALUES
(2, 'Amit', 22, 'BCA'),
(3, 'Rohit', 21, 'MCA');
```

2. READ Operation (SELECT)

The **READ** operation is used to **fetch data** from the database.

Read all records

```
SELECT * FROM students;
```

Read specific columns

```
SELECT name as “Std Name”, course FROM students;
```

Read with condition (WHERE)

```
SELECT * FROM students WHERE age > 22;
```

Read with sorting

```
SELECT * FROM students ORDER BY age DESC;
```

3. UPDATE Operation

The **UPDATE** operation is used to **modify existing records**.

Update single record

```
UPDATE students
SET course = 'M.Tech'
WHERE id = 1;
```

Update multiple columns

```
UPDATE students
SET age = 24, course = 'MBA'
WHERE name = 'Neeraj';
```

Important Note: Always use **WHERE clause**, otherwise **all records will be updated**.

4. DELETE Operation

The **DELETE** operation is used to **remove records** from a table.

Delete specific record

```
DELETE FROM students WHERE id = 3;
```

Delete all records

```
DELETE FROM students;
```

This will delete all rows but keep the table structure.

DELETE vs TRUNCATE

DELETE	TRUNCATE
Deletes selected rows	Deletes all rows
WHERE clause allowed	WHERE not allowed
Slower	Faster
Can be rolled back	Cannot be rolled back

```
TRUNCATE TABLE students;
```

CRUD Using Primary Key (Best Practice)

```
-- Create
INSERT INTO students VALUES (4, 'Rahul', 20, 'BSc');

-- Read
SELECT * FROM students WHERE id = 4;

-- Update
UPDATE students SET age = 21 WHERE id = 4;

-- Delete
DELETE FROM students WHERE id = 4;
```

What are Constraints in MySQL?

Constraints are rules applied to table columns to ensure the **accuracy, validity, and integrity of data** in a database.

They control **what type of data can be stored** in a table.

Types of Constraints in MySQL

1. NOT NULL

- Ensures that a column **cannot have NULL (empty) values**.

Example:

```
CREATE TABLE students (
    id INT NOT NULL,
    name VARCHAR(50) NOT NULL,
    age INT
);
```

`id` and `name` must have values
NULL values are not allowed

2. UNIQUE

- Ensures that **all values in a column are different**.

Example:

```
CREATE TABLE users (
    email VARCHAR(100) UNIQUE
);
```

No two users can have the same email

3. PRIMARY KEY

- A combination of **NOT NULL + UNIQUE**
- Uniquely identifies each record in a table
- Only **one primary key per table**

Example:

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50)
);
```

4. FOREIGN KEY

- Creates a relationship between **two tables**
- Ensures referential integrity

Example:

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
```

```
);
```

`customer_id` must exist in `customers` table

5. CHECK

- Ensures that column values satisfy a **specific condition**

Example:

```
CREATE TABLE students (
    age INT CHECK (age >= 18)
);
```

Only age 18 or above is allowed

6. DEFAULT

- Sets a **default value** if no value is provided

Example:

```
CREATE TABLE accounts (
    status VARCHAR(20) DEFAULT 'Active'
);
```

If status is not given, it becomes **Active**

Summary of Constraints

Constraint	Purpose
NOT NULL	Prevents NULL values
UNIQUE	Prevents duplicate values
PRIMARY KEY	Uniquely identifies records
FOREIGN KEY	Links two tables
CHECK	Applies a condition
DEFAULT	Sets default value

AND, OR, NOT Operators in MySQL

These are **logical operators** used in the `WHERE` clause to filter records.

1. AND Operator

- Returns records when **all conditions are true**

Syntax:

```
SELECT * FROM table_name
WHERE condition1 AND condition2;
```

Example:

```
SELECT * FROM students
WHERE age > 18 AND city = 'Lucknow';
```

Student must be **older than 18 AND from Lucknow**

2. OR Operator

- Returns records when **any one condition is true**

Syntax:

```
SELECT * FROM table_name
WHERE condition1 OR condition2;
```

Example:

```
SELECT * FROM students
WHERE city = 'Delhi' OR city = 'Mumbai';
```

Students from **Delhi OR Mumbai**

3. NOT Operator

- Reverses the condition
- Returns records when the condition is **false**

Syntax:

```
SELECT * FROM table_name
WHERE NOT condition;
```

Example:

```
SELECT * FROM students
WHERE NOT city = 'Lucknow';
```

Students **not from Lucknow**

Using AND, OR, NOT Together

Example:

```
SELECT * FROM employees
WHERE department = 'IT'
AND NOT salary < 30000;
```

IT employees with salary **30000 or more**

Operators: IN, LIKE, BETWEEN, NOT BETWEEN

These operators are used in the **WHERE clause** to filter data.

1. IN Operator

`IN` operator checks whether a value **matches any value in a given list**.

Multiple `OR` conditions ka short form hai.

Example

```
SELECT * FROM students
WHERE city IN ('Lucknow', 'Delhi', 'Mumbai');
```

Students from **Lucknow, Delhi or Mumbai**

NOT IN

```
SELECT * FROM students
WHERE city NOT IN ('Lucknow', 'Delhi');
```

Students **not from Lucknow and Delhi**

2. LIKE Operator

`LIKE` operator is used to **search patterns in text**.

Wildcards used in LIKE

Symbol	Meaning
`%`	Any number of characters
`_`	Exactly one character

Examples

Starts with letter

```
SELECT * FROM students
WHERE name LIKE 'A%';
```

Names starting with **A**

Ends with letter

```
SELECT * FROM students
WHERE name LIKE '%n';
```

Names ending with **n**

Contains word

```
SELECT * FROM students
WHERE name LIKE '%ram%';
```

Names containing **ram**

Exact length pattern

```
SELECT * FROM students
WHERE name LIKE '_a%';
```

Second letter **a**

NOT LIKE

```
SELECT * FROM students
WHERE name NOT LIKE 'A%';
```

3. BETWEEN Operator

`BETWEEN` operator selects values **within a given range (inclusive)**.

Range ke **start aur end dono include hote hain**.

Example (Numbers)

```
SELECT * FROM students
WHERE age BETWEEN 18 AND 25;
```

Age **18 se 25 ke beech (including 18 & 25)**

Example (Dates)

```
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

4. NOT BETWEEN Operator

Example

```
SELECT * FROM students
WHERE age NOT BETWEEN 18 AND 25;
```

Age **less than 18 OR greater than 25**

Quick Comparison Table

Operator	Use
IN	Multiple values match

Operator	Use
NOT IN	Exclude values
LIKE	Pattern search
NOT LIKE	Exclude pattern
BETWEEN	Range (inclusive)
NOT BETWEEN	Outside range

ORDER BY Clause in MySQL

The `ORDER BY` clause is used to **sort the result set** in **ascending or descending order**.

ORDER BY ASC (Ascending)

- `ASC` means **ascending order**
- Default order in MySQL
- Numbers → **small to large**
- Text → **A to Z**

Syntax

```
SELECT * FROM table_name
ORDER BY column_name ASC;
```

Example

```
SELECT * FROM students
ORDER BY name ASC;
```

Student names sorted from **A to Z**

ORDER BY DESC (Descending)

What is DESC?

- `DESC` means **descending order**
- Numbers → **large to small**
- Text → **Z to A**

Syntax

```
SELECT * FROM table_name
ORDER BY column_name DESC;
```

Example

```
SELECT * FROM students
ORDER BY age DESC;
```

Students sorted by **highest age first**

ORDER BY with Multiple Columns

Example

```
SELECT * FROM students
ORDER BY city ASC, age DESC;
```

First sort by **city (A–Z)**
If city same, then sort by **age (high to low)**

IS NULL and IS NOT NULL in MySQL

- `NULL` means **no value / missing value**
- It is **not zero**, **not empty string**

IS NULL

Used to check whether a column value is **NULL**.

Syntax

```
SELECT * FROM table_name
WHERE column_name IS NULL;
```

Example

```
SELECT * FROM students
WHERE email IS NULL;
```

Students whose **email is not provided**

IS NOT NULL

Used to check whether a column value is **NOT NULL**.

Syntax

```
SELECT * FROM table_name
WHERE column_name IS NOT NULL;
```

Example

```
SELECT * FROM students
WHERE phone IS NOT NULL;
```

Students whose **phone number exists**

Combined Example (ORDER BY + IS NOT NULL)

```
SELECT * FROM students
WHERE email IS NOT NULL
```

```
ORDER BY name ASC;
```

Only students with email
Sorted by name A–Z

Quick Comparison Table

Clause / Operator	Use
ORDER BY ASC	Sort ascending
ORDER BY DESC	Sort descending
IS NULL	Check missing value
IS NOT NULL	Check available value

LIMIT Clause in MySQL

The `LIMIT` clause is used to **restrict the number of rows** returned by a query.

Syntax

```
SELECT * FROM table_name
LIMIT number;
```

Example

```
SELECT * FROM students
LIMIT 5;
```

Shows **only first 5 records**

LIMIT with ORDER BY

```
SELECT * FROM students
ORDER BY marks DESC
LIMIT 3;
```

Shows **top 3 students by marks**

OFFSET in MySQL

The `OFFSET` clause is used to **skip a specific number of rows** before starting to return rows.

Mostly used in **pagination** (page-wise data).

Syntax

```
SELECT * FROM table_name
LIMIT limit OFFSET offset;
```

Example

```
SELECT * FROM students
LIMIT 5 OFFSET 5;
```

Skips first **5 records**
Shows **next 5 records**

Pagination Example

```
-- Page 1
SELECT * FROM students LIMIT 5 OFFSET 0;

-- Page 2
SELECT * FROM students LIMIT 5 OFFSET 5;

-- Page 3
SELECT * FROM students LIMIT 5 OFFSET 10;
```

Shortcut Syntax (MySQL)

```
SELECT * FROM students
LIMIT offset, limit;
```

Example:

```
SELECT * FROM students
LIMIT 5, 5;
```

Skip 5 rows, show next 5 rows

What are Aggregate Functions?

Aggregate functions perform **calculations on multiple rows** and return **a single result**.

They are commonly used with `SELECT` and `GROUP BY`.

Common Aggregate Functions

1. COUNT()

Use

Returns the **number of rows**

Syntax

```
SELECT COUNT(column_name) FROM table_name;
```

Example

```
SELECT COUNT(*) FROM students;
```

Total number of students

2. SUM()

Use

Returns the **total sum** of a numeric column

Example

```
SELECT SUM(marks) FROM students;
```

Total marks of all students

3. AVG()

Use

Returns the **average value**

Example

```
SELECT AVG(marks) FROM students;
```

Average marks

4. MIN()

Use

Returns the **minimum value**

Example

```
SELECT MIN(age) FROM students;
```

Youngest student age

---

5. MAX()

Use

Returns the **maximum value**

Example

```
SELECT MAX(marks) FROM students;
```

Highest marks

Aggregate Functions with WHERE

```
SELECT AVG(marks)
FROM students
WHERE city = 'Lucknow';
```

Average marks of students from Lucknow

Aggregate Functions with GROUP BY

```
SELECT city, COUNT(*)
FROM students
GROUP BY city;
```

Number of students in each city

Aggregate Functions with HAVING

```
SELECT city, COUNT(*)
FROM students
GROUP BY city
HAVING COUNT(*) > 5;
```

Cities having more than 5 students

Important Rules (Exam Points)

- Aggregate functions **ignore NULL values**
- `WHERE` is used **before grouping**
- `HAVING` is used **after GROUP BY**
- `LIMIT` is used at the **end of query**

Quick Summary Table

Topic	Purpose
LIMIT	Restrict number of rows
OFFSET	Skip rows
COUNT()	Count rows
SUM()	Total
AVG()	Average
MIN()	Lowest value
MAX()	Highest value

1. Primary Key (PK) in MySQL

A **Primary Key** is a column (or group of columns) that **uniquely identifies each record (row)** in a table.

Key Features of Primary Key

- It must be **UNIQUE**
- It **cannot be NULL**
- Only **one primary key** is allowed per table
- It helps to identify records clearly

Example: Students Table

```
CREATE TABLE students (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

Here:

- `roll_no` is the **Primary Key**
- Each student must have a **unique roll number**

Valid Data Example

roll_no	name	age
1	Rahul	20
2	Neha	21

Duplicate roll numbers are **not allowed**

2. Foreign Key (FK) in MySQL

A **Foreign Key** is a column in one table that **refers to the Primary Key of another table**.

It is used to **create a relationship between two tables**.

Key Features of Foreign Key

- It can have **duplicate values**
- It can be **NULL**
- It ensures **data consistency**
- It connects two tables

Example: Cities and Students Tables

Parent Table (Cities)

```
CREATE TABLE cities (
```

```
    city_id INT PRIMARY KEY,
    city_name VARCHAR(50)
);
```

Child Table (Students)

```
CREATE TABLE students (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    city_id INT,
    FOREIGN KEY (city_id) REFERENCES cities(city_id)
);
```

Here:

- `city_id` in **cities** = Primary Key
- `city_id` in **students** = Foreign Key

Sample Data

Cities Table

city_id	city_name
101	Lucknow
102	Delhi

Students Table

roll_no	name	city_id
1	Aman	101
2	Riya	102

`city_id` in students must exist in cities table

Important Note (Very Common Doubt)

Primary Key and Foreign Key values do NOT need to be same as roll numbers

Example:

- Student roll_no → `1, 2, 3`
- City city_id → `101, 102, 103`

This is **completely correct**